

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO2 – Manipulation (BL3, BL4)	Assessment:	Assessment Tool 1 – Analytical Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO2 Statement:	Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques.		
Student Name:	M.Dhanush Kumar	Reg. No.:	192S2S318
Section / Batch:		Date:	20/07/2026

Code of Conduct

I M.Dhanush Kumar / 192S2S318 (Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: M.Dhanush Kumar

Instructions

- This test contains 5 Analytical problem-solving questions. Total: 100 Marks.
- When a student answer using an Analytical problem-solving, in evaluation the answer should be with following five requirements: Understanding of problem, select appropriate data structure, Design the solution, analyze, Clarity of representation.
- Each student should write 2 questions. No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

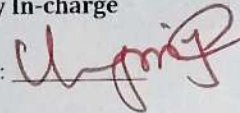
Section / Topic	Focus	Q Nos	Marks
Linked Data Structure, Search Data Structure	List Operations, Binary Search	1	50
Stack Data Structure, Queue Data Structures	Stack Notations, Priority Queue	2	50
GRAND TOTAL:			100 Marks

38	192524251	VAISHNAV M	- Two algorithms solve the same problem. Algorithm A uses recursion, while Algorithm B uses an explicit stack. Compare both approaches in terms of memory usage, execution flow, and risk of stack overflow. - A stack has a capacity of 5 elements. Explain what happens if the following operations are executed: Push(A), Push(B), Push(C), Push(D), Push(E), Push(F). Identify the error and suggest two possible solutions.
39	192525187	VEERA SANTHOSH A	- Compare the performance of a linear queue and a circular queue when 100 enqueue and dequeue operations are performed alternately. - A queue implemented using an array report "Queue Full" even though some positions are empty. - Analyze why this occurs. - Explain how a circular queue solves this problem.
40	192511226	VINIL KUMAR S	- Consider the singly linked list: $10 \rightarrow 20 \rightarrow 30 \rightarrow 40 \rightarrow 50$. Delete the node containing 30. Explain how the links change after deletion. - Given two sorted linked lists: L1: $1 \rightarrow 3 \rightarrow 5 \rightarrow 7$ L2: $2 \rightarrow 4 \rightarrow 6 \rightarrow 8$

Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20	Demonstrates complete and precise understanding; identifies all constraints and edge cases	Shows clear understanding with minor gaps	Partial understanding; misses some constraints	Misinterprets problem or lacks clarity
Data Structure Selection	20	Selects optimal data structure with strong justification and comparison	Selects appropriate structure with reasonable justification	Selects somewhat suitable structure with weak reasoning	Incorrect or no data structure selection
Algorithm Design	20	Designs efficient, logically sound, and well-structured algorithm	Algorithm is correct with minor inefficiencies	Algorithm is partially correct or lacks clarity	Algorithm is incorrect or missing
Accuracy of Solution	30	Error-free, well-structured, optimized code/solution	Minor errors but overall functional	Multiple errors; partially working	Non-functional or missing solution
Clarity	10	Exceptionally clear, well-organized, uses diagrams effectively	Clear and organized with minor issues	Somewhat organized but lacks clarity	Poorly organized and unclear

Faculty In-charge

Signature: 

Date: _____

Course Coordinator

Signature: _____

Date: _____

Program Director

Signature: _____

Date: _____

1) Count odd and Even Elements in a Singly Linked List.

Sol) $77 \rightarrow 88 \rightarrow 22 \rightarrow 11 \rightarrow 66 \rightarrow 27 \rightarrow 55 \rightarrow 65$
 $\rightarrow 18 \rightarrow 10 \rightarrow \text{NULL}$

Algorithm:

1. Start from the first node of the Linked List.
2. Set $\text{odd} = 0$ and $\text{even} = 0$.
3. Traverse each node until NULL.
4. If the node value is divisible by 2, increment even.
5. Otherwise, increment odd.
6. Finally, print the odd and even counts.

C-Program:

```
#include <stdio.h>  
#include <stdlib.h>
```

```
struct Node {  
    int data;  
    struct Node * Next;  
};
```

```

int main () {
    struct Node *head = NULL, *temp,
    *NewNode;

    int a[] =
    { 77, 88, 22, 11, 66, 27, 55, 65, 18, 10 };

    int odd = 0, even = 0;

    for (int i = 0; i < 10; i++) {
        NewNode = (struct
        Node *) malloc (sizeof (struct Node));
        NewNode -> data = a[i];
        NewNode -> next = NULL;
        if (head == NULL)
            head = NewNode;
        else {
            temp = head;
            while (temp -> next != NULL)
                temp = temp -> next;
            temp -> next = NewNode;
        }
    }
    temp = head;

```

```

temp = temp -> next;
}
}

temp = head;
while (temp != NULL) {
    if (temp -> data % 2 == 0)
        even++;
    else
        odd++;
    temp = temp -> next;
}

printf("odd count = %d\n", odd);
printf("Even count = %d\n",
even);

return 0;
}

```

2) Linear Search for Key $K = 20$

Given List

77 → ~~88~~ → 22 → 11 → 26 → 27 → ~~55~~ → 65 → 18 → 10

Algorithm:

1. Start from the first node.
2. Take the search key $K = 20$.

3. Compare the key with each node's data.
4. If the key matches any node, Print "YES".
5. If the complete list is searched and the key is not found, print "No".

C-Program:-

```
#include <stdio.h>
```

```
int main() {
```

```
    int a[] =
```

```
    {77, 88, 22, 11, 66, 27, 55, 65, 18, 10};
```

```
    int key = 20;
```

```
    int found = 0;
```

```
    for (int i = 0; i < 10; i++) {
```

```
        if (a[i] == key) {
```

```
            found = 1;
```

```
            break;
```

```
        }
```

```
    }
```

```
    if (found)
```

```
        printf("YES\n");
```

```
    return 0;
```

```
}
```

Output

No.